

第五课预习报告

add_one.c

- 作用: 演示前缀递增和后缀递增操作符的效果
- 功能: 使用 `super++` 和 `++ultra` 常量循环递增两个变量并输出它们的值
- 演示效果:

```
1  /* add_one.c -- incrementing: prefix and postfix */
2  #include <stdio.h>
3  int main(void)
4  {
5      int ultra = 0, super = 0;
6
7      while (super < 5)
8      {
9          super++;
10         ++ultra;
11         printf("super = %d, ultra = %d \n", super, ultra);
12     }
13
14     return 0;
15 }
```

```
1  super = 1, ultra = 1
2  super = 2, ultra = 2
3  super = 3, ultra = 3
4  super = 4, ultra = 4
5  super = 5, ultra = 5
```

addemup.c

- 作用: 演示 C 语言中五种不同类型的语句，包括声明、赋值、循环、函数调用以及返回语句
- 功能: 循环求前 20 个正整数的和并输出
- 演示效果:

```

1  /* addemup.c -- five kinds of statements */
2  #include <stdio.h>
3  int main(void)          /* finds sum of first 20 integers */
4  {
5      int count, sum;      /* declaration statement      */
6
7      count = 0;           /* assignment statement      */
8      sum = 0;             /* ditto                      */
9      while (count++ < 20) /* while                    */
10         sum = sum + count; /*      statement          */
11     printf("sum = %d\n", sum); /* function statement      */
12
13     return 0;            /* return statement        */
14 }

```

```

1  0 bottles of spring water!

```

wheat.c

- 作用: 演示指数级增长的概念
- 功能: 模拟国际象棋棋盘上每格小麦数翻倍, 并计算总数和占世界小麦产量的比例
- 演示效果:

```

1  /* wheat.c -- exponential growth */
2  #include <stdio.h>
3  #define SQUARES 64          // squares on a checkerboard
4  int main(void)
5  {
6      const double CROP = 2E16; // world wheat production in wheat grains
7      double current, total;
8      int count = 1;
9
10     printf("square    grains    total    ");
11     printf("fraction of \n");
12     printf("          added    grains    ");
13     printf("world total\n");
14     total = current = 1.0; /* start with one grain */
15     printf("%4d %13.2e %12.2e %12.2e\n", count, current,
16         total, total/CROP);
17     while (count < SQUARES)
18     {

```

```

19     count = count + 1;
20     current = 2.0 * current;
21     /* double grains on next square */
22     total = total + current;    /* update total */
23     printf("%4d %13.2e %12.2e %12.2e\n", count, current,
24            total, total/CR0P);
25 }
26 printf("That's all.\n");
27
28 return 0;
29 }

```

square	grains added	total grains	fraction of world total
1	1.00e+00	1.00e+00	5.00e-17
2	2.00e+00	3.00e+00	1.50e-16
3	4.00e+00	7.00e+00	3.50e-16
4	8.00e+00	1.50e+01	7.50e-16
5	1.60e+01	3.10e+01	1.55e-15
6	3.20e+01	6.30e+01	3.15e-15
...	(中间省略)		
64	9.22e+18	1.84e+19	9.22e+02

That's all.

squares.c

- 作用: 输出前20个整数及其平方的表格
- 功能: 使用 `while` 循环递增整数并计算其平方值, 按格式输出
- 演示效果:

```

1  /* squares.c -- produces a table of first 20 squares */
2  #include <stdio.h>
3  int main(void)
4  {
5      int num = 1;
6
7      while (num < 21)
8      {
9          printf("%4d %6d\n", num, num * num);
10         num = num + 1;
11     }

```

```
12
13     return 0;
14 }
```

1	1	1
2	2	4
3	3	9
4	4	16
5	5	25
6	6	36
7	7	49
8	8	64
9	9	81
10	10	100
11	11	121
12	12	144
13	13	169
14	14	196
15	15	225
16	16	256
17	17	289
18	18	324
19	19	361
20	20	400

sizeof.c

- 作用: 演示 `sizeof` 运算符的用法
- 功能: 使用 `sizeof` 获取 `int` 类型的字节大小, 并输出结果
- 演示效果:

```
1 // sizeof.c -- uses sizeof operator
2 // uses C99 %z modifier -- try %u or %lu if you lack %zd
3 #include <stdio.h>
4 int main(void)
5 {
6     int n = 0;
7     size_t intsize;
8
9     intsize = sizeof (int);
10    printf("n = %d, n has %zd bytes; all ints have %zd bytes.\n",
```

```

11         n, sizeof n, intsize );
12
13     return 0;
14 }

```

1 | `n = 0`, `n` has 4 bytes; all ints have 4 bytes.

shoes2.c

- 作用: 计算多种鞋码对应的脚长, 并以表格形式输出
- 功能: 使用 `while` 循环逐步增加鞋码, 计算每个鞋码的脚长
- 演示效果:

```

1  /* shoes2.c -- calculates foot lengths for several sizes */
2  #include <stdio.h>
3  #define ADJUST 7.31          // one kind of symbolic constant
4  int main(void)
5  {
6      const double SCALE = 0.333; // another kind of symbolic constant
7      double shoe, foot;
8
9      printf("Shoe size (men's)    foot length\n");
10     shoe = 3.0;
11     while (shoe < 18.5)        /* starting the while loop */
12     {                          /* start of block          */
13         foot = SCALE * shoe + ADJUST;
14         printf("%10.1f %15.2f inches\n", shoe, foot);
15         shoe = shoe + 1.0;
16     }                          /* end of block          */
17     printf("If the shoe fits, wear it.\n");
18
19     return 0;
20 }

```

1	Shoe size (men's)	foot length
2	3.0	8.31 inches
3	4.0	8.64 inches
4	5.0	8.97 inches
5	6.0	9.31 inches
6	7.0	9.64 inches

7	8.0	9.97 inches
8	9.0	10.31 inches
9	10.0	10.64 inches
10	11.0	10.97 inches
11	12.0	11.31 inches
12	13.0	11.64 inches
13	14.0	11.97 inches
14	15.0	12.30 inches
15	16.0	12.64 inches
16	17.0	12.97 inches
17	18.0	13.30 inches
18	If the shoe fits, wear it.	

shoes1.c

- 作用: 将男鞋尺码转换为英寸长度
- 功能: 使用符号常量和缩放比例常量, 计算鞋码对应的脚长
- 演示效果:

```

1  /* shoes1.c -- converts a shoe size to inches */
2  #include <stdio.h>
3  #define ADJUST 7.31           // one kind of symbolic constant
4  int main(void)
5  {
6      const double SCALE = 0.333; // another kind of symbolic constant
7      double shoe, foot;
8
9      shoe = 9.0;
10     foot = SCALE * shoe + ADJUST;
11     printf("Shoe size (men's)    foot length\n");
12     printf("%10.1f %15.2f inches\n", shoe, foot);
13
14     return 0;
15 }
```

1	Shoe size (men's)	foot length
2	9.0	10.31 inches

running.c

- 作用: 将公里制比赛的成绩转换为英里配速和平均速度
- 功能: 输入公里距离及跑步用时 (分钟和秒), 计算对应的英里配速和平均时速
- 演示效果:

```
1 // running.c -- A useful program for runners
2 #include <stdio.h>
3 const int S_PER_M = 60;           // seconds in a minute
4 const int S_PER_H = 3600;         // seconds in an hour
5 const double M_PER_K = 0.62137; // miles in a kilometer
6 int main(void)
7 {
8     double distk, distm; // distance run in km and in miles
9     double rate;         // average speed in mph
10    int min, sec;         // minutes and seconds of running time
11    int time;             // running time in seconds only
12    double mtime;         // time in seconds for one mile
13    int mmin, msec;       // minutes and seconds for one mile
14
15    printf("This program converts your time for a metric race\n");
16    printf("to a time for running a mile and to your average\n");
17    printf("speed in miles per hour.\n");
18    printf("Please enter, in kilometers, the distance run.\n");
19    scanf("%lf", &distk); // %lf for type double
20    printf("Next enter the time in minutes and seconds.\n");
21    printf("Begin by entering the minutes.\n");
22    scanf("%d", &min);
23    printf("Now enter the seconds.\n");
24    scanf("%d", &sec);
25    time = S_PER_M * min + sec; // converts time to pure seconds
26    distm = M_PER_K * distk;    // converts kilometers to miles
27    rate = distm / time * S_PER_H; // mph
28    mtime = (double) time / distm; // time per mile
29    mmin = (int) mtime / S_PER_M; // find whole minutes
30    msec = (int) mtime % S_PER_M; // find remaining seconds
31    printf("You ran %1.2f km (%1.2f miles) in %d min, %d sec.\n",
32           distk, distm, min, sec);
33    printf("That pace corresponds to running a mile in %d min, ",
34           mmin);
35    printf("%d sec.\nYour average speed was %1.2f mph.\n", msec,
36           rate);
37
38    return 0;
```

```
39 | }
```

```
1 | This program converts your time for a metric race
2 | to a time for running a mile and to your average
3 | speed in miles per hour.
4 | Please enter, in kilometers, the distance run.
5 | Next enter the time in minutes and seconds.
6 | Begin by entering the minutes.
7 | Now enter the seconds.
8 | You ran 10.00 km (6.21 miles) in 42 min, 30 sec.
9 | That pace corresponds to running a mile in 6 min, 50 sec.
10 | Your average speed was 8.77 mph.
```

rules.c

- 作用: 验证运算符优先级在表达式求值中的作用
- 功能: 使用复杂的运算组合计算结果, 并将同一结果赋值给两个变量
- 演示效果:

```
1 | /* rules.c -- precedence test */
2 | #include <stdio.h>
3 | int main(void)
4 | {
5 |     int top, score;
6 |
7 |     top = score = -(2 + 5) * 6 + (4 + 3 * (2 + 3));
8 |     printf("top = %d, score = %d\n", top, score);
9 |
10 |    return 0;
11 | }
```

```
1 | top = -23, score = -23
```

pound.c

- 作用: 定义一个接受整数参数的函数, 并根据参数值打印对应数量的字符 #
- 功能: 通过不同类型的参数 (int、char、float) 调用同一函数, 展示类型转换效果

- 演示效果:

```
1  /* pound.c -- defines a function with an argument */
2  #include <stdio.h>
3  void pound(int n);    // ANSI function prototype declaration
4  int main(void)
5  {
6      int times = 5;
7      char ch = '!';    // ASCII code is 33
8      float f = 6.0f;
9
10     pound(times);      // int argument
11     pound(ch);         // same as pound((int)ch);
12     pound(f);          // same as pound((int)f);
13
14     return 0;
15 }
16
17 void pound(int n)      // ANSI-style function header
18 {                     // says takes one int argument
19     while (n-- > 0)
20         printf("#");
21     printf("\n");
22 }
```

```
1  #####
2  #####
3  #####
```

post_pre.c

- 作用: 演示后缀递增 (postfix) 和前缀递增 (prefix) 的区别
- 功能: 比较 `a++` 与 `++b` 在赋值阶段的不同效果
- 演示效果:

```
1  /* post_pre.c -- postfix vs prefix */
2  #include <stdio.h>
3  int main(void)
4  {
5      int a = 1, b = 1;
```

```

6     int a_post, pre_b;
7
8     a_post = a++; // value of a++ during assignment phase
9     pre_b = ++b;  // value of ++b during assignment phase
10    printf("a  a_post  b  pre_b \n");
11    printf("%1d %5d %5d %5d\n", a, a_post, b, pre_b);
12
13    return 0;
14 }

```

```

1  a  a_post  b  pre_b
2  2    1    2    2

```

min_sec.c

- 作用: 将秒数转换为分钟与秒数
- 功能: 输入秒数, 程序计算对应的分钟和剩余秒数, 直到输入小于等于0结束
- 演示效果:

```

1  // min_sec.c -- converts seconds to minutes and seconds
2  #include <stdio.h>
3  #define SEC_PER_MIN 60          // seconds in a minute
4  int main(void)
5  {
6      int sec, min, left;
7
8      printf("Convert seconds to minutes and seconds!\n");
9      printf("Enter the number of seconds (≤0 to quit):\n");
10     scanf("%d", &sec);          // read number of seconds
11     while (sec > 0)
12     {
13         min = sec / SEC_PER_MIN; // truncated number of minutes
14         left = sec % SEC_PER_MIN; // number of seconds left over
15         printf("%d seconds is %d minutes, %d seconds.\n", sec,
16               min, left);
17         printf("Enter next value (≤0 to quit):\n");
18         scanf("%d", &sec);
19     }
20     printf("Done!\n");
21
22     return 0;

```

```

1  Convert seconds to minutes and seconds!
2  Enter the number of seconds (≤0 to quit):
3  125 seconds is 2 minutes, 5 seconds.
4  Enter next value (≤0 to quit):
5  70 seconds is 1 minutes, 10 seconds.
6  Enter next value (≤0 to quit):
7  Done!

```

golf.c

- 作用: 演示变量的多重赋值及格式化输出对齐效果
- 功能: 将三个球员分数赋值为相同值并以表格形式显示
- 演示效果:

```

1  /* golf.c -- golf tournament scorecard */
2  #include <stdio.h>
3  int main(void)
4  {
5      int jane, tarzan, cheeta;
6
7      cheeta = tarzan = jane = 68;
8      printf("           cheeta   tarzan   jane\n");
9      printf("First round score %4d %8d %8d\n",cheeta,tarzan,jane);
10
11     return 0;
12 }

```

```

1  cheeta   tarzan   jane
2  First round score   68       68       68

```

divide.c

- 作用: 演示整数除法、浮点除法以及混合类型除法的差异
- 功能: 通过直接表达式计算并打印不同类型的除法结果
- 演示效果:

```

1  /* divide.c -- divisions we have known */
2  #include <stdio.h>
3  int main(void)
4  {
5      printf("integer division: 5/4 is %d \n", 5/4);
6      printf("integer division: 6/3 is %d \n", 6/3);
7      printf("integer division: 7/4 is %d \n", 7/4);
8      printf("floating division: 7./4. is %1.2f \n", 7./4.);
9      printf("mixed division: 7/4 is %1.2f \n", 7./4);
10
11     return 0;
12 }

```

```

1  integer division: 5/4 is 1
2  integer division: 6/3 is 2
3  integer division: 7/4 is 1
4  floating division: 7./4. is 1.75
5  mixed division: 7/4 is 1.75

```

convert.c

- 作用: 演示 C 语言自动类型转换及赋值顺序
- 功能: 展示 `char`、`int`、`float` 类型赋值和运算时的类型提升与截断规则
- 语义错误: 第 15 行将 1107 赋值给 `char`，发生溢出，值从 1107 转换为 83 ('S')，虽可运行但会触发编译器警告
- 演示效果:

```

1  /* convert.c -- automatic type conversions */
2  #include <stdio.h>
3  int main(void)
4  {
5      char ch;
6      int i;
7      float fl;
8
9      fl = i = ch = 'C'; /* line 9 */
10     printf("ch = %c, i = %d, fl = %2.2f\n", ch, i, fl); /* line 10 */
11     ch = ch + 1; /* line 11 */
12     i = fl + 2 * ch; /* line 12 */

```

```

13     fl = 2.0 * ch + i;                                /* line 13 */
14     printf("ch = %c, i = %d, fl = %2.2f\n", ch, i, fl); /* line 14 */
15     ch = 1107;                                         /* line 15 */
16     printf("Now ch = %c\n", ch);                       /* line 16 */
17     ch = 80.89;                                        /* line 17 */
18     printf("Now ch = %c\n", ch);                       /* line 18 */
19
20     return 0;
21 }

```

```

1  ch = C, i = 67, fl = 67.00
2  ch = D, i = 203, fl = 339.00
3  Now ch = S
4  Now ch = P

```

bottles.c

- 作用: 循环演示“100瓶泉水”歌词，每次递减瓶数并输出相应内容
- 功能: 使用 `#define MAX 100` 定义常量，循环输出每一轮的瓶数，并模拟取下一瓶的过程
- 演示效果:

```

1  #include <stdio.h>
2  #define MAX 100
3  int main(void)
4  {
5      int count = MAX + 1;
6
7      while (--count > 0) {
8          printf("%d bottles of spring water on the wall, "
9                "%d bottles of spring water!\n", count, count);
10         printf("Take one down and pass it around,\n");
11         printf("%d bottles of spring water!\n\n", count - 1);
12     }
13
14     return 0;
15 }

```

```
1 100 bottles of spring water on the wall, 100 bottles of spring water!
2 Take one down and pass it around,
3 99 bottles of spring water!
4
5 ... (中间省略)
6 1 bottles of spring water on the wall, 1 bottles of spring water!
7 Take one down and pass it around,
8 0 bottles of spring water!
```